Student Referral Sheet – Faculty of Computing (Final Examination Jan - June 2026)

Student ID :

Module Code : SE3030

Lecture 1 - Software Architecture Overview

Software architecture is not only a folder structure, a framework, or a diagram. It is the set of important structural decisions that shape the system. We can describe architecture as a combination of the system structure or architectural style, the quality attributes the system must support, key architectural decisions, and design principles. A system can satisfy all functional requirements and still be architecturally weak if it is slow, insecure, unavailable, or hard to change.
Architectural Style: The main structure or pattern used to organize a software system. It shows how major parts of the system are separated and connected.
Example: A school LMS may use **layered architecture**: UI layer, service layer, database layer.
Quality Attributes / ASRs: The qualities that describe how well the system works, beyond just having features. These help judge whether the system is good, reliable, and usable.
Examples: Performance, security, availability, scalability, modifiability.
Architecture Decisions: Important technical rules or choices that guide how developers build the system. These decisions create boundaries and keep the system consistent.
Examples: "Only the service layer can access the database." UI pages cannot directly run SQL queries.
Design Principles: General guidelines that help developers make better design choices. They are flexible and guide the implementation, but they are not strict rules.
Examples: "Prefer asynchronous messaging between services to improve performance."
Software Architect Role: The person responsible for the system's technical direction. The architect makes key decisions, selects suitable structures, considers quality attributes, and guides the team.
A software architect is expected to: Make key technical decisions, Analyze and improve the architecture continuously, Stay updated with new technology trends, Ensure developers follow architecture rules, Use experience from different systems, Understand the business domain
Examples: The architect decides the system structure, security rules, database access rules, and quality goals.

Lecture 2 - Architectural Activities and Design Process

Architecture is created from **business decisions + technical decisions + social influences**. It is influenced by stakeholders, organization, architect's experience, and technical environment. After the architecture is used, it also affects future systems, teams, requirements, and processes. This is called the **Architecture Business Cycle (ABC)**.
1. Architecture Business Cycle — ABC
Stakeholders: People who affect architecture decisions.
Examples: Management wants low cost, marketing wants fast release, users want good UX & security.
Developing Organization: Company resources, structure & past investments affect architecture.
Examples: If company already uses Java/Spring Boot, architect may select Java-based architecture.
Architect's Experience: Architect's technical skills and domain knowledge affect decisions.
Examples: An architect experienced in microservices may suggest service-based architecture.
Technical Environment: Available tools, frameworks, processes, and engineering practices.
Examples: Cloud, Docker, Kubernetes, databases, testing tools.
ABC simple meaning: Architecture is influenced by the environment. After the system is built, that architecture again changes the environment, future requirements, team structure and architect experience.
2. Ramifications of Architecture

Effect	Meaning	Example
Affects organization structure	Architecture can change how teams are arranged.	Microservices may need separate teams for each service.
Affects organization goals	Successful architecture may create new business opportunities.	A good e-commerce platform may lead to mobile app development.
Affects future requirements	Customers may request new features based on the current system.	After online reports, users may ask for dashboard analytics.
Affects architect	Architect gains experience from success/failure of design.	Slow system teaches architect to focus more on performance next time.
Affects process and culture	Architecture influences development methods.	Microservices may require DevOps, CI/CD & automated testing.

3. Architectural Activities
Create Business Case: Understand market, cost, target users, time, and limitations.
The architect should first create a business case for modernizing the current desktop-based sales system. The current system is difficult for sales staff to use and has compatibility problems with modern software and hardware. The business benefit of modernization is improved usability, easier maintenance, better scalability, and future support for third-party POS integration. The architect should also consider cost, deadline, budget limitations, and project risks.
Understand Requirements: Identify functional and non-functional requirements.
The architect should identify functional and non-functional requirements. Functional requirements include sales record management, user login, report generation, cloud access, and POS integration. Non-functional requirements include usability, scalability, maintainability, performance, availability, and security. Constraints include limited budget, strict deadline, and the need to evolve the system incrementally.
Create or Select Architecture: Choose suitable architecture to satisfy behavior & quality needs.
A modular monolithic or layered architecture is suitable for the initial version. This is because the company has budget and deadline constraints, so a complex microservices architecture may be risky at the beginning. A modular monolith allows clean separation of modules such as Sales, Reports, Users, and POS Integration while keeping deployment simple.
Document and Communicate Architecture: Clearly explain architecture to developers, testers, & managers.
The architect should document the architecture using diagrams and descriptions. The document should show modules such as User Management, Sales Records, Reports, POS Integration, Database, and Cloud Deployment. This helps developers understand module boundaries, testers understand integration points, and managers understand the modernization plan.
Analyze or Evaluate Architecture: Compare possible designs and choose the best one.
The architecture should be evaluated against important quality attributes such as usability, scalability, maintainability, and integration. The architect should check whether the system can support future POS integration, cloud deployment, and growing users. Risks such as budget overrun, data migration problems, and integration complexity should be identified early.
Implement Based on Architecture: Build the system according to selected architecture.
During implementation, developers should follow the selected architecture. For example, the UI should not directly access the database. Business logic should be placed in service modules. POS integration should happen through a separate integration module or API. This keeps the system clean and maintainable.
Ensure Conformance: Check whether actual implementation follows architecture rules.
The architect should ensure that the implementation follows the architecture rules. Code reviews, dependency checks, and architecture reviews should be done during development and maintenance. This prevents quick fixes from breaking the design and avoids architecture erosion.
4. Understanding Requirements
Functional Requirements: What the system must do. **Non-Functional Requirements:** How well the system must work. **Quality Attribute Scenarios:** A structured way to describe non-functional requirements. **Prototyping:** Build a small sample to understand requirements better. **Domain Modeling:** Understand the real-world business area.
5. Architectural Design Process
Feasibility: Find possible design ideas. **Preliminary Design:** Select and improve the best idea. **Detailed Design:** Define detailed components and relationships. **Planning:** Adjust design for development, deployment, maintenance, and retirement.
6. Potential Problems in Architectural Design Process
No feasible concept: If the designer cannot find a practical design idea, the project cannot move forward. **High complexity:** When the system becomes large and complex, one person may not be able to design everything correctly. **System-level design issues:** Standard design methods may not handle situations where many products or systems must work together. **Lack of experience:** If the designer does not have enough experience, wrong design decisions may be made. **Need alternative approaches:** When complexity increases, the team may need cyclic, parallel, adaptive, or incremental design methods instead of a simple linear process.
7. Alternative Design Strategies
Standard / Linear: Follow steps one after another. **Cyclic:** Go back to earlier stages when needed. **Parallel:** Explore multiple solutions at the same time. **Adaptive:** Decide next step based on current results. **Incremental:** Improve the existing design step by step.
8. Design Tools and Patterns
Abstraction: Focus on important ideas and hide unnecessary details. **Modularity:** Divide system into separate modules. **Separation of Concerns:** Each part handles one responsibility. **Layered Pattern:** System is divided into layers. **MVC:** Separates Model, View, and Controller. **Client-Server:** Client requests services from server.

Lecture 3 - Architectural Structures and Views

Architecture can be understood using different **structures** and **views**. A structure is the actual set of elements in the system, while a view is the documented representation used by stakeholders.
Structure: The actual software/hardware elements and relationships in the system. (Actual modules: Login, Payment, Report.)
View: A diagram/document that represents a structure for stakeholders. (Class diagram, layered diagram, deployment diagram.)
1. Architectural Considerations: Architecture can be studied as code units, runtime elements, or relation to external environments.
Code organization: How the system is divided into code units. (Modules, packages, classes.)
Runtime behavior: How executing parts interact. (Components,services,connectors,processes.)
External environment: How software maps to hardware, files, network, or teams. (Server deployment, file structure, team assignment.)
2. Main Software Structures: Module Structures: Code-time organization. **Use:** Maintainability, planning, information hiding. **Component-and-Connector Structures:** Runtime behavior and communication. **Use:** Performance, availability, interaction analysis. **Allocation Structures:** Mapping software to environment. **Use:** Deployment, project management, implementation planning.
3. Module Structures: Decomposition: Breaks large modules into smaller modules. **Uses:** Shows which module depends on another. **Layered:** Organizes modules into layers. **Class / Generalization:** Shows inheritance or object relationships.
4. Component-and-Connector Structures: Process: Shows running processes/threads and communication. **Concurrency:** Shows parts that run in parallel. **Shared Data / Repository:** Components access common persistent data. **Client-Server:** Clients request services from servers.
5. Allocation Structures: Allocation structures map software to external environments. **Deployment:** Maps software components to hardware or servers. **Implementation:** Maps software modules to files/folders. **Work Assignment:** Assigns modules to teams.
6. Relating Software Structures: Different structures are connected, but they show different concerns.
Example: In an online ticketing system: Module structure: Booking module, Payment module, Notification module. **Component-and-connector structure:** Booking service communicates with payment gateway at runtime. **Allocation structure:** Booking service is deployed on app server, database on DB server.
7. What Structures Should Be Documented: It is not practical to document every structure. The architect should document the most useful structures based on the system's quality goals. **Performance:** Process, deployment, client-server views. **Security:** Deployment, layered, shared data views. **Maintainability:** Decomposition, layered, implementation views. **Scalability:** Client-server, process, deployment views. **Project management:** Work assignment view. **Dominant structure:** The most important structure for achieving the system's main requirements.
Example: For a high-traffic e-commerce system, deployment and client-server views may be dominant.
8. Views: A view is a documented representation of a structure. Different stakeholders need different views.
Architect: Deployment, layered, component views (To reason about quality attributes.) **Developer:** Class, module, implementation views (To understand code structure.) **Tester:** Component, process, deployment views (To plan integration and performance tests.) **Project Manager:** Work assignment, implementation views (To manage teams and schedule.) **Maintainer:** Module, layered, deployment views (To understand change impact.) **Customer / End User:** Overview view (To understand system capabilities.)
9. 4+1 View Model: The 4+1 view model describes architecture from different stakeholder viewpoints.
Logical View: Shows main functionality and object model. (Student, Teacher, Course, Enrollment classes.) **Development View:** Shows code/module organization. (Packages, modules, layers.) **Process View:** Shows runtime processes and communication. (Payment process, booking process, notification process.) **Physical View:** Shows deployment on hardware/network. (Web server, database server, mobile client.) **Scenarios / Use Cases:** Connects all views using real user actions. (Customer books a ticket and receives QR code.)

Lecture 4 - Choosing Between Monolithic and Distributed Architecture Styles

How to decide whether a system should be monolithic or distributed. The main idea is: architecture decisions are driven by **quality attributes**, not only functional requirements. If the whole system can share one set of quality attributes, monolithic is usually suitable. If different parts need different quality attributes, distributed architecture is usually better.
1. Modularity: Dividing the system into smaller understandable parts. (Payment module, user module, report module.) **Goal:** Make system easier to understand, change, test, and maintain. (Change payment logic without changing report module.)
2. Cohesion, Coupling, Connascence
Cohesion: How closely related things inside a module are. (High cohesion is good.) **Coupling:** How much one module depends on another module. (Low coupling is good.) **Connascence:** When two parts must change together because they depend on the same detail. (Lower connascence is better.)
3. Architecture Components: Module: Logical code unit, usually seen during design /development. **Component:** Real implementation/deployable unit of modules. **Architect's focus:** Components are important because architecture is implemented through components.
4. Architecture Partitioning: Partitioning means how we divide the system into major parts. Technical Partitioning: Divide by technical layers. (UI, business logic, data layer.) **Advantage:** Clear separation of code. **Disadvantage:** Business workflow may spread across many layers. Technical partitioning often leads to **layered architecture**. It is easy to understand but can create coupling across layers.
Domain Partitioning: Divide by business domain/workflow. (Order, Payment, Delivery, Customer.) **Advantage:** Matches business better and supports future distributed migration. **Disadvantage:** Shared technical concerns like logging/security need coordination. Domain partitioning is useful for **modular monoliths and microservices**, because each domain can later become an independent service.
5. Architecture Quantum: An independently deployable unit with high functional cohesion, synchronous connascence, and one uniform set of quality attributes.
6. Monolithic vs Distributed Architecture:

Aspect	Monolithic	Distributed
Deployment	Single deployment unit	Multiple deployment units
Quality attributes	One common set	Different per service/quantum
Complexity	Lower operational complexity	Higher operational complexity
Scalability	Whole system scales together	Parts scale independently
Failure isolation	Lower	Better isolation
Network issues	No internal network calls	Network latency/failure possible
Best for	Small team, fast delivery, simple system	Large system, independent scaling, team autonomy

System can operate with a single quality attribute: choose monolithic/modular monolith. **Different parts need different quality attributes:** choose distributed/microservices/event driven.
7. Complexity Cost of Distribution: Network latency: Remote calls are slower than local calls. **Reliability issues:** Network/service failure can break communication. **Security complexity:** More communication paths need protection. **Data consistency:** Harder when each service owns data. **Testing/debugging difficulty:** Failures spread across many services. **Operational cost:** Needs DevOps, monitoring, deployment automation.
10. Selecting the Appropriate Architecture

Scenario Clue	Better Choice	Reason
Small team, low budget, fast release	Modular Monolith	Simple and fast to build.
One common database and same quality needs	Monolithic	One quantum is enough.
Different modules need independent scaling	Distributed / Microservices	Multiple quanta.
Many async workflows and integrations	Event-driven	Loose coupling.
Future growth but early simplicity needed	Modular Monolith first, Microservices later	Reduces early risk.
High team autonomy needed	Microservices / domain partitioning	Teams own separate services.

Exam Answer Template: I would first identify the quality attributes and architecture quanta of the system. If the whole system can share one set of quality attributes and can be deployed as one unit, a monolithic or modular monolithic architecture is suitable. It reduces cost, complexity, and delivery risk. However, if different parts of the system require independent scalability, deployment, availability, or security, then the system has multiple architecture quanta and a distributed architecture is more suitable. The trade-off is that distributed architecture improves scalability and team autonomy but increases network, data consistency, testing, and operational complexity.

Lecture 5 - Architecture in the Real World: Reasoning from Examples

1. Architectural Reasoning: For any real-world system, ask these 3 questions:
What quality attributes are prioritized: Performance? Scalability? Availability? Modifiability?
What trade-offs are visible: What quality improves and what quality becomes harder? **What architectural structure reflects this:** Monolith? Microservices? Event-driven? Hybrid?

Lecture 6 - Software Quality Attributes

1. Software Requirements
Stakeholder requirements: Different people expect different things from the system. **Needs vs Wants:** Needs are essential; wants are optional/nice to have. **Requirement timing:** Requirements should be identified early. **Responsible people:** Business Analyst + Software Architect.
2. Types of Requirements
Functional Requirements: What the system must do. (User can book a ticket. Admin can generate report.) **Non-Functional Requirements: Quality Attributes:** How well the system must work. (Fast, secure, available, scalable.) **Business Requirements:** Business/strategic goals. (Low cost, fast time to market, long system lifetime.) **Constraints:** Fixed decisions with no freedom to change. (Must use open-source software only.)
3. Quality Attributes Overview: Quality attributes are measurable and testable properties used to judge how well the system satisfies stakeholder needs.
Availability: System services are available when and where users need them. (Online banking should be available 24/7.)
Affected by system errors, maintenance, infrastructure failures, malicious attacks, system load, and dependent services. It is often measured using uptime percentage such as 99.9% availability.
Interoperability: Ability to exchange and understand information with other systems. (Hospital system connects with lab system and insurance system.)
Two parts: **Syntactic interoperability:** systems use compatible formats/protocols, **Semantic interoperability:** systems understand the meaning of exchanged data.
Modifiability: Cost and effort needed to make changes. Lower change cost = higher modifiability. (Add a new payment method without changing many modules.)
Change types: Functional change, Environment/platform change, Protocol change, Dependency change, such as database change
Performance: Responsiveness of the system within a time interval. (Search results appear within 1 second.)
Measured by: Latency: time taken to respond, Throughput: no of requests processed per time.
Reliability: Ability of a system to remain operational and perform correctly over time. (Payment system does not fail during transactions.) Affected by availability, accuracy, and predictability.
Reusability: Components/subsystems can be reused in other applications or situations. (A login library reused in multiple systems.) **Benefit:** reduces duplication and development time.
Scalability: Ability to handle increased load without major performance loss. (E-commerce site handles Black Friday traffic.)
Types: Load scalability: more users/requests, Functional scalability: more features, Geographic scalability: more regions/locations.
Methods: Horizontal scaling: add more servers, Vertical scaling: improve one server's hardware.
Trade-off: Horizontal scaling improves capacity but increases management and communication complexity.
Security: Prevent attacks, unauthorized access, and protect system assets. (Encryption, authentication, authorization.) A Denial of Service attack is not only a security issue. It can also affect availability, performance, and usability.
Testability: Ability to create and run tests to check whether the system meets criteria. (Payment module can be unit tested independently). Good testability helps faults be isolated quickly and effectively.
Usability: How easily and effectively users can use the system. (Simple checkout, clear error messages, undo option.)
Usability considers: How easy it is to learn, How efficiently users can use it, How well it handles user errors, How well it adapts to user needs, How much confidence users have in system actions.
4. Architectural Attributes and Considerations: Architectural quality attributes are about the architecture itself, not only the final system.
Conceptual Integrity: Similar things should be done in similar ways, **Correctness and Completeness:** Architecture should not have errors or missing parts, **Buildability:** The available team can complete the system in time and adapt to changes.
5. Role of Architect in Quality Attributes
Understand priorities: Identify which quality attributes matter most, **Identify ASRs:** Find architecturally significant requirements, **Balance needs and wants:** Not every stakeholder wish can be satisfied fully, **Evaluate trade-offs:** Improve important qualities while managing negative effects.

Lecture 7 - Quality Attribute Scenarios (QAS)

Quality attributes can be vague unless written as scenarios. QAS is a formal way to write quality requirements, so they become measurable and testable. A general scenario can apply to any system, while a concrete scenario is specific to the given system. Concrete QAS is very useful in exams because it shows that you can convert "should be fast" or "should be reliable" into a measurable architectural requirement.
Source: Who trigeers? | **Stimulus:** What event? | **Environment:** Under what condition | **Artifact:** Which part? | **Response:** System reaction | **Measure:** Metric / target
1. Availability QAS
Source: External system, **Stimulus:** Sends unexpected message, **Environment:** Normal operation, **Artifact:** Main process, **Response:** System records the event, notifies operator, continues operation, Response **Measure:** No downtime
Concrete example: A payment gateway becomes unavailable during normal checkout. The order system detects the failure, records the error, notifies support, and allows the customer to retry or use another payment method within 1 minute.
2. Modifiability QAS
Source: Developer, **Stimulus:** Wants to add new payment method, **Environment:** Design/build time, **Artifact:** Payment module, **Response:** Change is made without affecting order and report modules, **Response Measure:** Completed and tested within 2 days
Concrete example: A developer adds a new SMS notification provider during build time. The notification module is updated through an interface without changing booking or payment modules, and the change is completed within 1 day.
3. Performance QAS
Source: Customer, **Stimulus:** Submits booking request, **Environment:** Peak traffic, Artifact: Booking service, **Response:** System confirms or rejects booking, **Response Measure:** Response within 3 seconds
Concrete example: During a popular concert ticket launch, a customer submits a seat booking request. The booking service checks seat availability and returns confirmation or rejection within 3 seconds while preventing seat duplication.
4. Security QAS
Source: Unauthorized user / attacker, **Stimulus:** Attempts to access patient records, **Environment:** Normal operation, **Artifact:** Patient data service, **Response:** System blocks access, logs attempt, alerts admin, **Response Measure:** Access denied immediately; log created within 1 second
Concrete example: An unauthorized user attempts to access hospital patient records during normal operation. The system denies access, records the attempt in the audit log, and notifies the security administrator within 5 seconds.
5. Testability QAS
Source: Tester, **Stimulus:** Runs automated test suite, **Environment:** Testing environment before release, Artifact: Payment module, Response: System executes tests and reports pass/fail results, Response Measure: 90% of tests complete within 10 minutes
Concrete example: A tester runs automated regression tests before deployment. The system executes all critical module tests and produces a report within 15 minutes with failed test details.
6. Usability QAS
Source: New user, **Stimulus:** Tries to complete registration, **Environment:** First-time use, **Artifact:** Registration UI, **Response:** System guides user and shows clear errors, **Response Measure:** User completes registration within 5 minutes without help
Concrete example: A first-time student uses the university registration portal during the enrollment period. The system provides clear instructions, validates errors, and allows the student to complete registration within 5 minutes.
7. How to Identify QAS from Case Study: "must be online 24/7": **Availability,** "respond quickly / high traffic": **Performance,** "future changes / new features": **Modifiability,** "sensitive data / unauthorized access": **Security,** "easy for users / simple UI": **Usability,** "many tests / safety critical": **Testability,** "must not lose data": **Reliability,** "growing users / peak load": **Scalability**

Lecture 8 - Quality Attribute Tactics

A tactic is an architectural design decision used to achieve a quality attribute response measure. Patterns may package multiple tactics, but tactics are more specific. For example, microservices may support modifiability and scalability, but caching specifically improves performance; heartbeat specifically detects failure for availability.
1. Availability Tactics: Availability tactics help keep the system running even when faults occur. A fault can become a failure if the system cannot handle it. Availability tactics try to **detect faults**, **recover from faults**, or **prevent faults.**
Fault Detection: Ping / Echo: One component sends a ping and waits for reply. **Heartbeat:** Component sends regular "I am alive" signals. **Exceptions:** System detects errors using exception handling.
Fault Recovery — Preparation and Repair: Voting: Multiple components produce results; majority result is accepted. **Active Redundancy:** All backup components run at the same time.

Passive Redundancy: Backup is updated but not active until main fails. Spare: Standby component replaces failed component.
Fault Recovery — Reintroduction: Removal from Service: Take component offline before it fails. Transactions: Group steps so they all succeed or all fail. Process Monitor: Monitor process and restart it if it fails.
2. Modifiability Tactics: Modifiability tactics reduce the time, cost, and risk of making changes. They focus on **localizing modifications**, **preventing ripple effects**, and **deferring binding time**.
Localize Modifications: Maintain Semantic Coherence: Keep related responsibilities in the same module. Anticipate Expected Changes: Design around likely future changes. Generalize Module: Make module flexible for multiple cases. Limit Possible Options: Reduce unnecessary variation. Abstract Common Services: Put common logic in reusable service.
Prevent Ripple Effects: Hide Information: Hide internal module details. Maintain Existing Interfaces: Keep interface stable even if implementation changes. Restrict Communication Paths: Limit unnecessary module communication. Use an Intermediary: Use mediator/message broker/API gateway.
Defer Binding Time: Configuration: Decide behavior using config files/settings. Plugins: Add features at runtime/build time. Runtime registration: Add or replace services dynamically.
3. Performance Tactics: Performance tactics aim to reduce response time, increase throughput, and manage resource usage.
Control Resource Demand: Manage Sampling Rate: Reduce how often events/data are processed. Limit Event Response: Ignore or limit low-priority events. Prioritize Events: Important tasks processed first. Reduce Overhead: Remove unnecessary processing. Bound Execution Time: Limit task running time. Increase Resource Efficiency: Use better algorithms/data structures.
Manage Resources: Concurrency: Run tasks in parallel. Replication: Use multiple copies of component/data. Caching: Store frequently used data closer. Load Balancing: Distribute requests across servers. Scheduling: Decide order of task execution.
4. Security Tactics: Security tactics protect the system from unauthorized access, attacks, and misuse.
Authenticate Users: Verify identity. Authorize Users: Check permissions. Encrypt Data: Protect data from being read. Limit Exposure: Reduce attack surface. Detect Attacks: Identify suspicious behavior. Audit / Logging: Record security-related actions. Recover from Attack: Restore safe state.
5. Usability Tactics: Usability tactics help users complete tasks easily and confidently.
Cancel: Let user cancel operation. Undo: Reverse previous action. Pause / Resume: Continue later. Maintain Task Model: System understands user task flow. Maintain User Model: System remembers user preferences/role. Provide Feedback: Tell user what is happening. Prevent Errors: Stop mistakes before they happen.
6. Testability Tactics: Testability tactics make it easier to test and find faults.
Record / Playback: Record events and replay them during testing. Separate Interface from Implementation: Test components through stable interfaces. Specialized Testing Interface: Add test hooks/interfaces. Built-in Monitor: System monitors internal state. Logging: Record events for debugging.
7. Tactic Selection by Scenario Clues

Scenario Clue	Quality Problem	Suitable Tactics
"System must not go down"	Availability	redundancy, heartbeat, failover
"High traffic / peak users"	Performance, scalability	caching, load balancing, replication
"Sensitive data"	Security	authentication, authorization, encryption, audit
"Future features expected"	Modifiability	modularity, abstraction, stable interfaces
"Users make mistakes"	Usability	undo, cancel, validation, feedback
"Need frequent testing"	Testability	logging, test interfaces, monitoring
"Avoid data loss"	Reliability	transactions, backup, checkpoint/rollback
"External service may fail"	Availability	retry,timeout, fallback, circuit breaker approach

Lecture 9 - Architectural Patterns and Styles

Architectural styles are reusable structural solutions for recurring architecture problems. They are not tied to a specific programming language or framework. In the 2026 exam announcement, practical applications and pros/cons of Monolithic, Modular Monolithic, Microkernel, Microservices, Event-driven, Component-based, and Layered styles are especially important.

Style	Best-fit context	Strengths	Weaknesses / trade-offs
Layered	Business applications with clear separation of UI, business logic, and data access.	Easy to understand, maintain, test, and assign responsibilities.	Can become rigid; lower layers cannot call upper layers without dependency problems; performance overhead if every request crosses many layers.
Monolithic	Small/simple applications with limited team and fast delivery needs.	Simple to develop, test, deploy, and operate; low initial complexity.	Hard to scale parts independently; large codebase can become difficult to maintain over time.
N-tier	When logical layers need physical separation or independent resource allocation.	Can scale tiers separately; improve deployment separation.	More deployment complexity and network communication cost.
Component-based	Systems built from reusable/ replaceable parts.	Reusability, substitutability, lower development cost.	Versioning, integration, and dependency management can become hard.
Object-Oriented	Systems that can be modeled as real-world objects with data and behavior.	Encapsulation, reusability, easier mapping to real-world entities.	Poor design can create tight coupling and complex object relationships.
Domain-Driven	Complex business systems where domain rules are very important.	Aligns software with business concepts; improves communication with domain experts.	Can become complex with large teams; needs strong domain understanding.
Client-Server	Centralized data/functionality accessed by many clients.	Central control, easier security, centralized maintenance.	Server bottleneck or single point of failure unless replicated.
Microkernel	Stable core with optional features/extensions.	Excellent extensibility and product-line support.	Plugin management, version compatibility, and core/plugin interface design are critical.
Microservices	Large evolving systems needing independent scaling /deployment.	Team autonomy, independent deployment, independent scaling.	Distributed complexity, data consistency, observability, network failure, DevOps cost.
Event-driven	Many components communicate asynchronously or need loose coupling.	Scalable, responsive, decoupled, good for notifications/integration	Harder debugging, eventual consistency, message ordering/duplication issues.
Modular Monolith	Initial release with one deployable but clean internal boundaries.	Simpler operations than microservices while supporting maintainability.	Requires discipline; can decay into big ball of mud.
Pipe and Filter	Data processing systems where data passes through sequential steps.	Filters are reusable; processing flow is clear; good for transformation pipelines.	Not ideal for interactive systems; performance can suffer if many filters are chained.
Cloud Architecture	Systems needing elastic scaling, global access, managed infrastructure, or pay-as-use resources.	Scalability, availability, flexible resources, reduced infrastructure management.	Vendor lock-in, cost control, security/compliance concerns.
Blackboard	Complex problem-solving systems where many independent components contribute partial solutions.	Good for AI, recognition, and decision-support systems; supports multiple knowledge sources.	Shared data coordination is difficult; can be hard to control and debug.

Simple CRUD / business app: **Layered,** Small team / fast release / low complexity: **Monolithic,** Physical layer separation / scalable web app: **N-tier,** Reusable replaceable parts: **Component-based,** Real-world objects/entities: **Object-Oriented,** Complex business domain: **Domain-Driven,** Many clients using central server: **Client-Server,** Plugins / optional extensions: **Microkernel,** Independent scaling/deployment: **Microservices,** Async events / notifications: **Event-driven,** One deployment with clean modules: **Modular Monolith,** Step-by-step data processing: **Pipe and Filter,** Elastic/global/cloud scaling: **Cloud Architecture,** Shared AI/problem-solving knowledge: **Blackboard**

Lecture 10 - Software Architecture Evaluation

Architecture evaluation checks whether the chosen architecture can satisfy the required quality attributes before the system becomes expensive to change. Evaluation can be early or late. Early evaluation is preferred because architecture changes are cheaper before implementation. Evaluation may use questioning, measurement, scenarios, mathematical models, simulation, or expert reasoning.
1. Why Evaluate Software Architecture: Check fitness: To see whether the architecture matches system goals. Find risks early: Problems are cheaper to fix before implementation. Validate quality attributes: Check performance, security, availability, modifiability, etc. Improve decisions: Helps architects compare architecture options. Avoid disaster: Poor architecture can cause high cost, delays, poor performance, and maintenance issues.
2. When to Evaluate Architecture: Early evaluation: Before implementation starts. During design: While architecture is being created. Late evaluation: After implementation or in legacy systems.
3. How to Evaluate Architecture: Qualitative evaluation: Uses expert judgment, scenarios, and discussions. Quantitative evaluation: Uses measurements, metrics, simulations, or mathematical models. Scenario-based evaluation: Uses real scenarios to test architecture decisions. Experience-based reasoning: Uses evaluator/architect experience to identify risks.
4. Evaluation Method Types: Scenario-based: Uses scenarios to check quality attributes. Mathematical model-based: Uses formulas/models. Simulation-based: Simulates architecture behavior. Experience-based reasoning: Uses expert review.
5. Challenges in Architecture Evaluation: Need expert evaluators: Evaluation needs skilled people. Unclear architecture understanding: Stakeholders may not understand the design equally. Different stakeholder interests: Users, managers, developers, and admins want different things. Incomplete quality requirements: NFRs may not be written clearly. Unpredictable risks: Some risks are hard to identify early.
6. Who is Involved: Evaluation Team: Conducts evaluation and analysis. Usually should be skilled and objective. Stakeholders: People affected by the system and architecture. Architects: Explain architecture decisions and constraints. Project Decision Makers: People who can approve changes or investments.
7. Planning an Evaluation: Clear goals: Know what quality attributes to evaluate. Evaluation method: Choose ATAM, SAAM, ARID, etc. Controlled scope: Focus on most important goals only. Key stakeholders: Include representatives of important groups. Competent evaluation team: Preferably separate from developers. Managed expectations: Explain what evaluation can and cannot do.
8. Outputs of Architecture Evaluation: Prioritized quality attributes: Most important quality goals ranked. Mapping of approaches to qualities: Shows how architecture decisions support or fail quality attributes. Risks: Potentially problematic architecture decisions. Non-risks: Good decisions based on valid assumptions. Sensitivity points: Decisions that strongly affect one quality attribute. Trade-off points: Decisions that affect multiple qualities positively/negatively.
9. Advantages of Architecture Evaluation: Forces clear quality goals: Stakeholders must define what they really need. Prioritizes conflicting goals: Helps decide what matters most. Brings stakeholders together: Improves communication and agreement. Improves documentation: Architecture becomes clearer. Finds reuse opportunities: Similar solutions can be reused. Reduces later cost: Early fixes are cheaper than late fixes.
10. Architecture Evaluation Methods:
ATAM — Architecture Tradeoff Analysis Method: Main use: Evaluates multiple quality attributes and trade-offs. Best for: Large/complex systems with competing qualities. Focus: Business drivers, quality scenarios, risks, sensitivity points, trade-off points.
Example: For an online banking system, ATAM can analyze trade-offs between security, performance, availability, and usability.
SAAM — Software Architecture Analysis Method: Main use: Scenario-based architecture evaluation. Best for: Modifiability and comparing candidate architectures. Focus: How scenarios affect architecture components.
Example: For an LMS, SAAM can evaluate how difficult it is to add SMS notifications or a new report module.
ARID — Active Reviews for Intermediate Designs: Main use: Reviews partial/intermediate designs. Best for: Subsystems, components, or designs before full implementation. Focus: Whether the design is usable and suitable for stakeholders.
Example: Review the payment module design before implementing the full e-commerce platform.
CBAM — Cost Benefit Analysis Method: Main use: Evaluates economic impact of architecture decisions. Best for: Choosing between architecture options based on cost and benefit. Focus: Business value, cost, benefits, return on investment.
Example: Compare whether adding redundancy is worth the extra infrastructure cost.
11. Method Selection Clues: Multiple quality attributes conflict: ATAM, Need trade-off analysis: ATAM, Need modifiability/change scenario evaluation: SAAM, Need to compare candidate architectures using scenarios: SAAM, Need to review partial design/subsystem: ARID, Need cost-benefit decision: CBAM, Legacy system evaluation after implementation: Late evaluation + metrics / SAAM / ATAM

Lecture 11 - Trade-off Analysis and ATAM

A trade-off means improving one quality may reduce another. ATAM provides a structured method to reason about these conflicts using business drivers, scenarios, architectural approaches, and analysis. It helps identify sensitivity points where small changes strongly affect a quality attribute, and trade-off points where one decision affects multiple quality attributes positively or negatively.
1. Common Quality Attribute Trade-offs

Decision / Improvement	Improves	May Reduce
Strong authentication	Security	Usability, performance
Caching	Performance	Data consistency
Redundancy	Availability, reliability	Cost, complexity
Microservices	Scalability, modifiability, team autonomy	Cost, complexity
Event-driven communication	Scalability, loose coupling	Debugging simplicity, consistency
Encryption	Security	Performance
Strict consistency	Data correctness	Availability, performance
Extra logging/monitoring	Testability, maintainability	Performance, storage cost

2. What is ATAM: (Architecture Trade-off Analysis Method) Purpose: Evaluates whether an architecture fits multiple competing quality attributes. Main focus: Business drivers, quality attribute scenarios, risks, sensitivity points, trade-off points. Best use: Complex systems where performance, security, availability, modifiability, etc. conflict.
3. Why Use ATAM: Assess design decisions: Checks consequences of architecture choices. Clarify quality attributes: Converts stakeholder needs into scenarios. Find trade-offs: Shows where one quality improves while another reduces. Find risks early: Identifies weak architecture decisions before implementation. Support stakeholder agreement: Brings stakeholders together to discuss priorities.
4. Quality Attributes and ATAM: Security vs Performance: Encryption and access checks may slow response time. Security vs Usability: Multi-factor login improves security but adds user steps. Availability vs Consistency: Distributed systems may stay available but show slightly stale data. Performance vs Modifiability: Highly optimized code may be harder to change. Scalability vs Simplicity: Microservices scale better but are more complex.
5. Stakeholder Expectations: End users: Performance, availability, usability, security, Developers: Maintainability, modifiability, testability, reusability, Business / Management: Cost, time to market, benefits, target market, system lifetime, Operations / Admins: Reliability, security, monitoring, rollback, availability.
6. Architect's Role in Trade-off Analysis: Envision: Understand future system direction and quality goals. Realize: Convert architecture ideas into practical structures and decisions. Influence: Guide stakeholders, teams, and technical decisions.
7. Challenges in Trade-off Analysis: Quality attributes are vague: "Fast" or "secure" must be made measurable. Attributes conflict: Improving one can reduce another. Need early analysis: Hard to know suitability without building system. Stakeholder priorities differ: Users, developers, and business people want different things. Architecture complexity: Complex systems are harder to evaluate.
Participants of ATAM: Evaluation Team, Project Decision Makers, Architecture Stakeholders
Phases of ATAM: Phase 0 - Preparation: Prepare evaluation team, architecture documents, stakeholders, and scenarios. Phase 1- Evaluation Team Review: Evaluation team studies business drivers, architecture, quality goals, and architectural approaches. Phase 2- Stakeholder Review: Wider stakeholders give scenarios, prioritize concerns, and discuss risks/trade-offs. Phase 3- Follow-up: Issues are addressed and final report is prepared
ATAM Phase 1 Main Steps: 1. Present ATAM: Explain the evaluation process and rules. 2. Present Business Drivers: Explain business goals, constraints, important functions, and quality needs. 3. Present Architecture: Architect explains the proposed architecture and major decisions. 4. Identify Architectural Approaches: Identify patterns, tactics, styles, and key design decisions. 5. Generate Utility Tree: Organize and prioritize quality attributes and scenarios. 6. Analyze Approaches: Check how architecture decisions support or damage quality attributes.
Utility Tree: A structure used to organize quality attributes and scenarios. Purpose: Helps identify the most important quality requirements. Prioritization: Scenarios are ranked by business importance and technical risk/difficulty.
Example: Utility → Performance → "Search results within 2 seconds during peak load." Utility → Security → "Unauthorized users cannot access patient data."
ATAM Outputs: Risks: Architecture decisions that may cause problems. Non-risks: Good decisions based on correct assumptions. Sensitivity Points: Decisions that strongly affect one quality attribute. Trade-off Points: Decisions that affect multiple quality attributes. Prioritized Scenarios: Most important quality scenarios ranked by stakeholders. Final Report: Evaluation results, risks, and recommendations.
Sensitivity Point vs Trade-off Point: Sensitivity Point: A decision that strongly affects one quality attribute. Trade-off Point: A decision that affects more than one quality attribute.

Lecture 12 - Architecture Frameworks, Enterprise Architecture, and Cloud

1. Why Software Architecture is Needed: Standard governing structure, Solutions to known problems, Project success, Long-term maintainability, Risk reduction
2. When Do We Need to "Architect": Solution becomes large: Bigger systems need proper structure. System is complex: Many modules, users, and integrations. Need to think about future: Software is not thrown away; it must evolve. Increased usage types: Not only humans, but other systems interact.
3. Enterprise Architecture: A holistic practice for analyzing, designing, planning, and implementing enterprise systems. Purpose: Align business, information, process, and technology changes with organizational strategy. Focus: Not only software; includes business goals, data, processes, people, and technology.
4. Benefits of Enterprise Architecture
Business Benefits: Supports business strategy, Faster time to market, Consistent processes, better reliability and security.
IT Benefits: Better IT cost traceability, Lower IT cost, Faster development, Less complexity, Less IT risk.:
5. Key Architecture Principles: Build to change: Design for future changes, not only current needs. Model to analyze and reduce risk: Use diagrams/models to understand impact before building. Communication and collaboration: Use architecture views to explain decisions to stakeholders. Identify key engineering decisions: Focus on decisions that are costly to change later.
6. Enterprise Architecture Frameworks:
7.Zachman Framework: Classification scheme using What, How, Where, Who, When, Why.
A formal and structured way to view and define an enterprise. Purpose: Helps classify architecture information from different perspectives.
What: Data / things (Students, courses, payments), How: Processes/functions (Registration, enrollment, exams), Where: Locations/networks (Campuses, cloud servers), Who: People/roles (Students, lecturers, admins), When: Time/events (Registration period, exam period), Why: Goals/motivation (Reduce manual work, improve efficiency).

Row	Viewpoint	Meaning	Simple Example
Scope / Planner View	Planner	High-level business scope and purpose. Shows what the organization wants to achieve.	University wants a digital student management system.
Business Model / Owner View	Business Owner	Describes business processes, rules, people, and goals.	Admin manages students, lecturers manage courses, students register online.
System Model / Designer View	System Designer / Architect	Converts business needs into system architecture and models.	Design modules: student, course, payment, exam, reports.
Technology Model / Builder View	Developer / Engineer	Shows how the system will be built using technology and components.	Web app, database, APIs, authentication service.
Detailed Representations / Subcontractor View	Implementer	Detailed technical specifications for implementation.	Database tables, API endpoints, UI forms, configuration files.
Functioning Enterprise / User View	Actual System / User	The real working system in operation.	Students use the portal to register and view results.

8.TOGAF: (The Open Group Architecture Framework) Framework for designing, planning, implementing, and governing enterprise IT architecture. Purpose: Provides an approach for designing, planning, implementing, and governing enterprise IT architecture. Level of design: High-level enterprise architecture approach. Common architecture levels: Business, Application, Data, Technology.
9. TOGAF Architecture Levels: Business Architecture: Business goals, processes, and organization. Application Architecture: Software applications and their relationships. Data Architecture: Data entities, storage, and flow. Technology Architecture: Infrastructure, platforms, networks.
10. TOGAF Main Components: ADM - Architecture Development Method: Step-by-step process for developing or changing architecture. Enterprise Continuum: Organizes architecture assets, standards, and reusable solutions. Resource Base: Guidelines, templates, checklists, and supporting materials.
11. TOGAF ADM - Architecture Development Method: Preliminary Phase: Define how architecture work will be done; establish principles and framework. Phase A - Architecture Vision: Define high-level vision, goals, business value, stakeholders, and constraints. Phase B - Business Architecture: Define business processes, organization structure, and business capabilities. Phase C - Information Systems Architecture: Define data architecture and application architecture. Phase D - Technology Architecture: Define infrastructure, platforms, networks, and technology standards. Phase E - Opportunities and Solutions: Identify possible solutions and implementation options. Phase F - Migration Planning: Plan how to move from current architecture to target architecture. Phase G — Implementation Governance: Ensure implementation follows architecture. Phase H — Architecture Change Management: Manage changes after implementation. Requirements Management: Continuously manages requirements throughout ADM.
12. TOGAF vs Zachman

Aspect	TOGAF	Zachman
Type	Process/method framework	Classification/viewpoint framework
Main focus	How to develop and govern architecture	How to organize architecture information
Key idea	ADM cycle	What, How, Where, Who, When, Why
Use	Planning and implementing architecture change	Understanding enterprise from multiple perspectives
Best for	Enterprise architecture projects	Organizing enterprise knowledge/views

13. Architecture Frameworks in Case Studies: Large organization/enterprise system: **Enterprise Architecture**, Need business + IT alignment: **TOGAF**, Need structured architecture development: **TOGAF ADM**, Need multiple stakeholder viewpoints: **Zachman**, Need migration from old to new system: **TOGAF ADM** migration planning, need governance during implementation: **TOGAF Phase G**, Need long-term architecture change control: **TOGAF Phase H**

LECTURE COMPARISON

Lecture 1 - Software Architecture Overview: Define software architecture, explain architectural style, quality attributes, architecture decisions, design principles, or architect's role.
Lecture 2 - ABC + Architectural Activities: Explain Architecture Business Cycle, identify factors affecting architecture decisions, justify a business case, or list next architectural activities.
Lecture 3 - Structures and Views: Explain 4+1 View Model, explain module / component-and-connector / allocation structures, or explain an architecture diagram.
Lecture 4 - Monolithic vs Distributed: Case study question asking whether to choose monolith, modular monolith, microservices, or distributed architecture; justify trade-offs.
Lecture 6 - Quality Attributes / NFR: Identify functional and non-functional requirements, explain performance / availability / security / scalability, or select critical quality attributes.
Lecture 7 - QAS: Write Quality Attribute Scenarios using Source, Stimulus, Environment, Artifact, Response, Measure. Commonly availability or performance QAS.
Lecture 8 - Quality Attribute Tactics: Suggest tactics to improve availability, performance, security, modifiability, etc. Example: caching, load balancing, redundancy, authentication.
Lecture 9 - Architectural Patterns and Styles: Compare styles such as Layered, N-tier, Microservices, Event-driven, Microkernel, Modular Monolith, Cloud; recommend a suitable style for a case study.
Lecture 10 - Architecture Evaluation: Explain why architecture evaluation is needed, explain SAAM / ATAM / ARID / CBAM, or discuss architecture conformance.
Lecture 11 - Trade-off Analysis and ATAM: Explain trade-offs, sensitivity points, trade-off points, ATAM phases, ATAM outputs, or quality conflicts like performance vs consistency.
Lecture 12 - Enterprise Architecture + Cloud: Explain cloud architecture, compare IaaS / PaaS / SaaS, explain TOGAF / Zachman, or propose architecture for a large organization system.

When the question says **"draw high-level architecture / block diagram"**, include these 6 things: Users / Clients, API Gateway / UI Layer, Main Modules or Services, Database / Storage, External Systems, Communication method: REST API / Event Bus / Queue

High-Level Diagrams